# Invexa Retail Next

## Production Architecture Blueprint

A recommended foundation for a multi-tenant inventory management SaaS with advanced control systems, web operations, offline mobile workflows, and enterprise-grade engineering.

| | |
|---|---|
| **Scope** | Architecture recommendation only |
| **Direction** | New repository; existing Invexa remains unchanged |
| **Prepared** | 26 September 2026 |

# 1. Executive direction

**Recommended decision**

Build Invexa Next as a TypeScript-first modular monorepo: Next.js 16 for web surfaces, NestJS as a modular-monolith API, PostgreSQL plus Prisma for data, a dedicated worker for asynchronous workloads, and Flutter for the offline-first mobile client.

- Keep the current Invexa repository untouched as the reference implementation and migration source.
- Do not start with microservices. A modular monolith provides strong boundaries without multiplying deployment and operational cost.
- Build in complete vertical slices: contract, backend rules, database, UI states, telemetry, tests, and rollback together.
- Preserve proven foundations from the current product: immutable stock movements, idempotency, durable outbox, tenant scoping, auditability, and offline command contracts.
- Move away from large route files and duplicated frontend/admin foundations by defining enforceable module contracts from day one.

## What should stay

- PostgreSQL and Prisma
- Immutable inventory journal
- Outbox and background workers
- OpenAPI contract discipline
- Flutter and SQLite mobile foundation
- Cross-tenant negative testing
- Role-first operational UX

## What should change

- Vite web to Next.js 16
- Express route-heavy API to NestJS feature modules
- Separate, inconsistent admin UI to a deliberate control-plane surface
- Application-only tenant checks to application checks plus Postgres RLS
- Ad-hoc permissions to a centralized RBAC + ABAC policy model

# 2. Target repository and module structure

| Surface | Purpose | Primary boundary |
|---|---|---|
| apps/web | Tenant-facing operational dashboard | Next.js App Router; routes only in app/, domain code in modules |
| apps/control | Platform operations and high-privilege administration | Separate trust boundary and deployment from tenant UI |
| apps/api | Business API | NestJS modular monolith; no domain logic in controllers |
| apps/worker | Imports, outbox, sync, notifications, scheduled reports | Durable jobs; never block request paths |
| apps/mobile | Warehouse and field operations | Flutter, SQLite, offline command queue |
| packages/contracts | OpenAPI-derived clients and shared identifiers | Generated output; never hand-edited |
| packages/ui | Shared web-only design system | No forced web/mobile UI sharing |
| packages/authz | Permission vocabulary and policy contracts | One canonical policy decision surface |
| packages/config | Typed environment and build configuration | Validated at startup |
| packages/observability | Logs, metrics, tracing helpers | Common correlation and redaction rules |
| packages/testing | Fixtures and cross-surface test utilities | Unique tenant/user/resource IDs per run |

## Backend module contract

- Controller: transport mapping only.
- Application service: orchestration, transaction boundary, authorization call, and use-case result.
- Domain: invariants, lifecycle transitions, value objects, and policies without NestJS or Prisma imports.
- Repository: tenant-scoped persistence only; ORM rows never leak into domain contracts.
- Mapper: explicit conversion between database, API, and domain representations.
- Every dependency direction enforced by import-boundary lint and CI.

## Recommended bounded contexts

Identity, Organization, Catalog, Inventory, Warehouse, Transfers, Procurement, Orders, Returns, Stock Counts, Imports, Reporting, Notifications, Integrations, and Control Plane.

# 3. Data integrity, tenancy, and permissions

## PostgreSQL and inventory integrity

- Every tenant-owned row carries tenantId/companyId and an indexed tenant-leading access path.
- Enable and force PostgreSQL Row-Level Security for tenant tables; use a non-owner application role and transaction-local tenant context.
- Keep explicit tenant predicates in repositories; RLS is the final safety layer, not the only layer.
- Use NUMERIC/Decimal for quantity, price, tax, cost, and amount. Never introduce Float for exact business values.
- StockMovement remains append-only and authoritative. Mutable stock balance is a projection that can be reconciled.
- Every mutating inventory command uses a caller-stable idempotency key and an atomic unique barrier.
- Business change plus publish intent is one database transaction through the outbox pattern.
- Schema changes use migrations, backfills, compatibility windows, and rollback planning - never runtime DDL.

## Authorization model

**RBAC + ABAC**

RBAC supplies understandable role templates. ABAC narrows each decision by company, branch, warehouse, resource ownership, approval amount, operating hours, risk state, and temporary elevation. Default is deny; the backend is authoritative.

- Canonical permissions such as inventory.view, inventory.adjust, transfer.approve, purchase_order.release, user.manage, and audit.export.
- Maker-checker rules: users cannot approve their own operation; thresholds and segregation-of-duties are policy inputs.
- Temporary elevation expires automatically and requires a reason; break-glass access creates a high-severity audit event.
- Every allow and deny records subject, action, resource, tenant, policy version, reason, trace ID, and timestamp.
- Test both owning-tenant success and real-ID cross-tenant denial; conceal existence with 404 where appropriate.
- Quarterly access reviews, inactive-user detection, session/device revocation, and permission expiry are first-class workflows.

# 4. Web, control plane, and mobile experience

## Tenant web application

- Next.js Server Components by default; client components only at interactive leaves.
- Feature-oriented modules; app/ contains routing files, not business logic.
- TanStack Query owns client-side server state. React Hook Form plus Zod owns complex form state and validation.
- URL-owned filters, sorting, search, pagination, and saved views for operational tables.
- Shared data table, filter bar, drawer, form field, feedback states, confirmation and recovery patterns.
- Every workflow explicitly handles loading, empty, error, offline, permission-denied, partial-success, and conflict states.
- Permission-aware UI improves clarity, but never replaces server authorization.

## Platform control plane

- Tenants, plans, entitlements, usage limits, and feature rollout.
- Users, role templates, custom roles, permission matrix, access reviews, and policy simulator.
- Branch/location assignments, approval policies, active sessions, devices, forced logout, and security events.
- Integrations, webhook deliveries, job retries, system health, audit explorer, and support impersonation with explicit approval and audit.
- High-risk operations are reversible where possible and require typed confirmation, reason, and secondary approval where needed.

## Mobile application

> **Recommended mobile direction**
>
> Keep Flutter for Invexa Next unless a validated spike proves React Native materially better. Existing Flutter knowledge, barcode scanning, SQLite, and offline inventory workflows make Flutter the lower-risk fit.

- Feature-first layered architecture with presentation, application, domain, and data boundaries.
- SQLite local source, durable command queue, client-generated IDs, retry with backoff, and per-entity conflict policies.
- Sync checkpoints and resumable processing; background execution is a hint, never a schedule guarantee.
- Secure token storage, no secrets in the binary, and remote feature flags with a kill switch.
- Test on low-end Android hardware, weak networks, process death, permission denial, and interrupted scans.
- Crash reporting, ANR tracking, cold-start budget, frame-time monitoring, and staged store rollout.

# 5. Engineering quality, security, and operability

| Gate | Required evidence |
|---|---|
| Architecture | Import boundaries, no cycles, no new god files, generated contracts not hand-edited |
| Code quality | Lint, formatting, strict types, no dead code, narrow reviewed diff |
| Unit tests | Domain behavior, state transitions, value objects, permission decisions |
| Integration tests | Real PostgreSQL, migrations, concurrency, tenant isolation, outbox, idempotency |
| Component tests | Keyboard/touch access, semantic state, form validation, recovery states |
| E2E tests | Critical user journeys in a real browser and selected mobile device flows |
| Security | Threat model, dependency scan, secret scan, authz allow/deny cases, session replay tests |
| Release | Preview/staging proof, commit-SHA deploy, rollback workflow, smoke test |

## Observability baseline

- Structured JSON events with tenant ID, actor ID, trace ID, entry point, stable event name, and PII redaction.
- RED metrics for every endpoint and external dependency: request rate, error rate, and duration histograms.
- Queue metrics: pending count, oldest age, retries, poison/dead-letter count, and processing duration.
- Inventory correctness signals: ledger/projection drift, duplicate command rate, negative-stock blocks, and reconciliation failures.
- OpenTelemetry traces propagated through HTTP, jobs, outbox events, and webhook processing.
- Alerts on user symptoms and SLO burn, each linked to a tested runbook.

## CI/CD policy

- Every pull request runs lint, typecheck, unit, integration, contract, build, E2E where relevant, and security checks.
- Failing gates block merge; tests are fixed, not skipped.
- Production deploys are staged and reversible; feature flags separate deployment from release.
- Dependency and migration changes receive dedicated review and compatibility verification.

# 6. Recommended delivery sequence

| Phase | Outcome | Exit condition |
|---|---|---|
| 0 - Decisions | Architecture brief, glossary, domain boundaries, permission vocabulary, migration map | Every contract names an enforcing test or lint rule |
| 1 - Foundation | Monorepo, Next.js, NestJS, worker, Postgres, contracts, CI, telemetry | Health checks and empty vertical slice deploy successfully |
| 2 - Identity and tenancy | Sessions, tenant context, RLS, RBAC + ABAC, audit decisions | Cross-tenant and allow/deny suites pass |
| 3 - Inventory slice | Item search, stock detail, adjustment, reversal, reconciliation, web UI | Real-Postgres concurrency and idempotency tests pass |
| 4 - Mobile offline slice | Count/scan workflow, SQLite queue, sync, conflicts, recovery | Airplane-mode and process-kill scenarios pass |
| 5 - Control plane | Tenant/user/role/policy/session/audit and operational health surfaces | High-risk actions require reason, policy, and audit |
| 6 - Domain expansion | Transfers, procurement, orders, returns, imports, reporting | Each ships as a complete observable vertical slice |
| 7 - Migration and launch | Validated data migration, shadow reconciliation, pilot tenants, staged rollout | Rollback proven; SLOs healthy through pilot |

**First build recommendation**

Do not begin by copying every current feature. Start with the foundation, identity/tenancy, and one inventory vertical slice. Prove architecture, security, testing, observability, and delivery before expanding domain coverage.

## Decisions to freeze before implementation

- Tenant domain strategy and whether custom domains are part of the first release.
- One canonical company/tenant vocabulary across API, database, UI, mobile, and audit logs.
- Permission taxonomy and maker-checker approval boundaries.
- Flutter retained unless a time-boxed React Native spike beats it on offline database, scanner, performance, and maintainability.
- Migration policy: what data moves, what is archived, and how ledger totals are reconciled.
- Release target and rollback ownership for web, API, worker, database, and mobile.

## Current status

This document is a recommendation only. No source code, schema, repository structure, or deployment configuration was changed while preparing it.